

Self-Correcting RAG con Phoenix

MVP mínimo, narrativa de pitch y plan de sprints para el track de Arize

El pitch en una frase

“El agente convierte cada alucinación en un test de regresión.” Un RAG que no solo responde: detecta sus propios fallos con LLM-as-a-judge, los transforma en datasets de regresión vía Phoenix MCP, propone un prompt candidato, y deja que un humano lo promocione moviendo un tag. Observabilidad como capacidad operativa, no como dashboard pasivo.

Principios de diseño

- **Phoenix sí, AX no.** Una sola plataforma, una sola autenticación, un solo SDK.
- **RAG pequeño y curado.** El corpus es un entregable, no un detalle. Debe contener al menos un fallo claramente arreglable vía prompt.
- **El bucle es el producto.** Lo diferencial no es el RAG, es *fallo* → *dataset* → *prompt candidato* → *experimento* → *tag*.
- **Humano en el loop.** El agente nunca se auto-promociona a producción. Genera candidatos; el humano mueve el tag.
- **Desplegar el día uno.** Cloud Run vacío antes que features completas en local.

1. MVP mínimo

El MVP es el conjunto más pequeño de piezas que permite contar la historia completa del pitch de principio a fin. Si una pieza no aparece en la demo, no está en el MVP. Todo lo demás es capa opcional.

Qué entra en el MVP

- **Runtime ADK en Python** con un único *qa_agent* que responde sobre un corpus pequeño y curado.
- **Trazado completo a Phoenix Cloud** vía `openinference-instrumentation-google-adk` y `phoenix.otel.register()`.
- **Un solo evaluador: Faithfulness** sobre el span final. Es la métrica estrella de alucinación y la que sostiene el pitch.
- **Prompts servidos por Phoenix recuperados por tag** (production y candidate), nunca por nombre.
- **Phoenix MCP Server integrado en el agente** vía `McpToolset`, con un set mínimo de herramientas expuestas: leer trazas/spans, añadir ejemplo a dataset, `upsert` de prompt.
- **Un dataset de regresión** poblado a mano con 5–10 fallos representativos del corpus.
- **Un experimento comparativo** en Phoenix entre prompt production y prompt candidate sobre ese dataset.
- **Despliegue en Cloud Run** con imagen explícita (Python + Node para `npx @arizeai/phoenix-mcp`).

Qué NO entra en el MVP (capas opcionales)

- Multiagente (*qa_agent* + *critic_agent* separados). Empieza monolítico.
- Document Relevance y Tool Response Handling. Solo si Faithfulness ya está estable.
- Prompt Learning automático. Primero el bucle a mano funcionando, luego automatización.
- Vertex AI RAG Engine. Empieza con un retriever trivial (BM25 o embeddings locales) sobre 10–20 docs.
- Anotaciones humanas vía UI. Útiles para la narrativa pero no bloqueantes.
- Self-hosting de Phoenix. Phoenix Cloud para todo.

Criterio de éxito del MVP

Demo de 3 minutos, dos actos

Acto 1. Pregunta que el RAG contesta bien. Se enseña el trace limpio en Phoenix con spans de retrieve y modelo, y el score de Faithfulness en verde.

Acto 2. Pregunta diseñada para inducir alucinación. Faithfulness la marca en rojo. El agente — vía Phoenix MCP — consulta trazas similares, añade el caso al dataset de regresión, genera una versión candidate del prompt y la sube a Phoenix. Se lanza un experimento comparativo. Si mejora, se mueve el tag a staging. **Cierre:** “Cada alucinación se ha convertido en un test de regresión.”

2. Narrativa del pitch

Frase ancla

“El agente convierte cada alucinación en un test de regresión.”

Estructura de 90 segundos

Tiempo	Sección	Mensaje
0:00–0:15	Problema	Los RAGs alucinan. Los equipos descubren los fallos en producción, los parchean a mano, y rara vez convierten esos fallos en tests. La observabilidad se queda en dashboards pasivos.
0:15–0:30	Insight	La observabilidad puede ser una capacidad operativa del agente , no una pantalla. Si el agente puede leer sus propias trazas vía MCP, puede aprender de sus errores.
0:30–1:30	Demo	Dos actos. Respuesta buena con trace limpio. Respuesta alucinada que activa el bucle: Faithfulness lo marca, el agente consulta Phoenix MCP, crea un ejemplo de dataset, genera un prompt candidato, lanza un experimento, y un humano promociona el tag.
1:30–1:45	Diferencial	El humano siempre está en el loop. El agente propone , el humano promueve . Y todo deja traza auditada en Phoenix: el run malo, el dataset, el experimento y el evaluador.
1:45–2:00	Cierre	<i>“No solo responde. Aprende de sus propios errores bajo control humano.”</i>

Tres frases que un juez no técnico puede repetir

- *“Cada alucinación se convierte en un test de regresión.”*
- *“El agente propone, el humano promueve.”*
- *“La observabilidad deja de ser un dashboard y se convierte en una capacidad operativa.”*

3. Plan de sprints

Seis sprints cortos. Los cuatro primeros son el MVP. Los dos últimos son la capa que diferencia el proyecto del resto del track. Si el tiempo aprieta, los sprints 5 y 6 son sacrificables; los sprints 0 a 4 no lo son.

SPRINT 0 · Setup y despliegue vacío		día 1, mañana
Objetivo	Tener una app ADK trivial corriendo en Cloud Run, mandando trazas a Phoenix Cloud, antes de añadir nada de lógica. Validar el camino crítico de despliegue cuanto antes.	
Features	• Clonar el starter repo oficial del hackathon como base. • Cuenta en Phoenix Cloud + API key en variables de entorno. • phoenix.otel.register() + instrumentación ADK verificada con un hello-world. • Dockerfile con Python y Node (necesario para npx @arizeai/phoenix-mcp en sprint 4). • Despliegue en Cloud Run con health check público.	
Entregable	URL pública de Cloud Run que responde y deja un trace visible en Phoenix Cloud.	
Riesgos / mitigación	• Confundir Phoenix con AX: verificar endpoints y SDK desde el primer commit. • El buildpack por defecto de Cloud Run no instala Node. Imagen explícita desde el principio.	

SPRINT 1 · RAG mínimo con corpus curado		día 1, tarde
Objetivo	Un retriever trivial sobre un corpus pequeño, controlado y diseñado para tener fallos interesantes.	
Features	• Corpus de 10–20 documentos cortos sobre un dominio acotado (elegir uno donde tú seas capaz de juzgar la calidad). • Retriever simple: embeddings + similitud coseno, o BM25. Nada de Vertex RAG Engine. • Tool retrieve_documents conectada al qa_agent. • Prompt inicial almacenado en Phoenix bajo el tag production. • Corpus diseñado con fallos: al menos una ambigüedad terminológica, una pregunta con respuesta parcialmente en el corpus, y una pregunta sin respuesta para forzar abstención.	
Entregable	Agente que contesta preguntas con trace visible mostrando span de retriever + span de modelo.	
Riesgos / mitigación	• Corpus demasiado limpio: si no hay fallos arreglables vía prompt, el segundo acto de la demo se cae. • Curar el corpus es un entregable explícito, no un detalle.	

Objetivo	Activar el evaluador estrella y verlo discriminar respuestas buenas de respuestas alucinadas.
Features	• Evaluador Faithfulness preconfigurado de Phoenix sobre el span final de respuesta. • Pipeline de evaluación en línea (cada respuesta del agente queda evaluada). • Dashboard de Phoenix mostrando distribución de scores y top fallos. • Validación manual: 5 preguntas buenas y 5 diseñadas para alucinar deben separarse en el score.
Entregable	Captura de Phoenix mostrando claramente respuestas con Faithfulness alta vs baja.
Riesgos / mitigación	• Si Faithfulness no discrimina bien, el problema es el corpus, no el evaluador. Volver a sprint 1. • No añadir más evaluadores aún: tentación de complejidad prematura.

SPRINT 3 · Phoenix MCP integrado en el agente		día 2, tarde
Objetivo	Que el agente pueda leer sus propias trazas y escribir artefactos en Phoenix. Este es el sprint que convierte el proyecto en algo diferente al resto.	
Features	• McpToolset conectado al Phoenix MCP Server vía stdio dentro del contenedor. • Exposición mínima de herramientas: list_recent_traces, get_span, add_dataset_example, upsert_prompt. • Lógica condicional: si Faithfulness < umbral, el agente consulta MCP y añade el caso al dataset regression_v1. • Verificar en Phoenix que los ejemplos llegan al dataset con asociación al span original.	
Entregable	Una pregunta alucinada genera automáticamente una entrada en el dataset de regresión visible en Phoenix.	
Riesgos / mitigación	• Conexiones MCP persistentes complican el escalado: definir McpToolset de forma síncrona. • Filtrar bien qué herramientas se exponen: un MCP demasiado abierto es un riesgo de seguridad.	

SPRINT 4 · Prompt candidato y experimento comparativo		día 3, mañana
Objetivo	Cerrar el bucle: del dataset de regresión nace un prompt candidato, y un experimento decide si mejora.	
Features	• Generación de prompt candidato: una llamada al modelo con el prompt actual + ejemplos fallidos del dataset. • upsert_prompt del candidato con tag candidate en Phoenix. • Experimento en Phoenix (vía SDK o Playground) comparando production vs candidate sobre regression_v1. • Regla de promoción: si Faithfulness mejora y no degrada respuestas buenas, mover tag a staging. Manual. • Script promote_tag.py para que el humano lo lance en la demo.	
Entregable	Demo end-to-end funcionando: fallo → dataset → candidato → experimento → tag promovido.	
Riesgos / mitigación	• El prompt candidato puede no mejorar nada. Aceptable: la historia es el bucle, no el resultado. • Si el candidato mejora poco, ensayar la demo con un fallo que sí sea claramente prompt-fixable.	

SPRINT 5 · Capas opcionales y robustez		día 3, tarde
Objetivo	Solo si los sprints 0–4 están estables. Añadir las capas que suben la nota sin poner en riesgo el MVP.	
Features	• Document Relevance como segundo evaluador para distinguir fallos de recuperación vs generación. • Separar <i>qa_agent</i> y <i>critic_agent</i> para narrativa más fuerte de multiagente. • Cache local de prompts con fallback al último válido (mitiga riesgo de dependencia de red). • Tool Response Handling si el agente usa más herramientas. • Anotaciones humanas vía UI de Phoenix sobre los fallos detectados.	
Entregable	Versión enriquecida del MVP sin romper la demo principal.	
Riesgos / mitigación	• Sobrecargar la demo y perder claridad. Si una capa no aporta al pitch, fuera.	

SPRINT 6 · Pitch, demo y ensayo		día 4, mañana
Objetivo	El proyecto técnico está. Ahora gana o pierde el pitch.	
Features	• Guion de 90 segundos memorizado (no leído). • Demo grabada como fallback por si la conexión falla. • Las tres frases ancla preparadas: <i>“cada alucinación se convierte en un test de regresión”</i>, <i>“el agente propone, el humano promueve”</i>, <i>“observabilidad como capacidad operativa”</i>. • Slide con arquitectura visual: runtime ↔ Phoenix Cloud ↔ MCP ↔ dataset ↔ experimento. • Ensayo cronometrado al menos 3 veces.	
Entregable	Pitch ensayado y demo de respaldo lista.	
Riesgos / mitigación	• Falta de ensayo: la causa más común de perder un hackathon con un proyecto técnicamente bueno.	

4. Recordatorios finales

- **Curar el corpus es el sprint más infravalorado.** Sin fallos arreglables vía prompt, no hay segundo acto.
- **Desplegar el día uno.** Cloud Run vacío antes que features completas en local.
- **El humano siempre promueve.** No vendas “auto-mejora”, vende “mejora controlada”.
- **Una métrica estrella.** Faithfulness sostiene todo el pitch. El resto es decoración útil.
- **Si algo del sprint 5 amenaza al MVP, fuera del sprint 5.** La demo del MVP es la que gana.